

Assignment 1

Due Date: September 27,2026, 11:59pm

Instructions: Complete the assignment independently, as it is an individual task. Any instances of plagiarism will be met with strict consequences. Ensure to submit coding assignments in **Python (Jupyter Notebook)** by the due date. Each challenge should be a separate cell in the notebook for readability. Carefully follow all the provided guidelines.

Section 1

Code Challenge 1: Implement pattern count function. Find the number of times a Pattern occurs in Text string.

Input: Strings *Text* and *Pattern*.

Output: *Count(Text, Pattern)*.

```
# Create your pattern_count function, along with any subroutines you need
def pattern_count(text: str, pattern: str) -> int:
    pass
```

Sample Input:

ACGTACGTACGT
CG

Sample Output:

3

Code Challenge 2: Solve the Frequent Words Problem. Given a string *Text* and an interger *k*, find all most frequent *k*-mers (*k*-sized patterns) in *Text*. You may create any supporting funtions/subroutines for your code. Note that there can be more than one most frequent patterns so your routine must return a list of patterns.

Input: A string *Text* and an integer *k*.

Output: All most frequent *k*-mers in *Text*.

```
# Create your frequent_words function here, along with any subroutines you need
def frequent_words(Text: str, k: int) -> list[str]:
    pass
```

Sample Input:

CGTTTTGAACATTTTCAACAAGTTTTGCAACATTTT
4

Sample Output:

TTTT

Code Challenge 3: Solve the Minimum Skew Problem.

Minimum Skew Problem: Find all positions in a genome where the skew diagram attains a minimum.

Input: A DNA string *Genome*.

Output: All integer(s) i minimizing $Skew_i(Genome)$ among all values of i (from 0 to $|Genome|$).

Create your MinimumSkew function, along with any subroutines you need

```
def minimum_skew(genome: str) -> list[int]:  
    pass
```

Sample Input:

CCGGCCGG

Sample Output:

2 6

Code Challenge 4: Solve the Frequent Words with Mismatches Problem. Find the most frequent k -mers with up to d mismatches in a text.

Input: A string *Text* as well as integers k and d . (You may assume $k \leq 12$ and $d \leq 3$.)

Output: All most frequent k -mers with up to d mismatches in *Text*.

Create your frequent_words_with_mismatches function, along with any subroutines you need

```
def frequent_words_with_mismatches(text: str, k: int, d: int) ->  
list[str]:  
    pass
```

Sample Input:

ACGTTGCATGTCGCATGATGCATGAGAGCT
4 1

Sample Output:

ATGT GATG ATGC

Bonus Challenge: Find a *DnaA* box in *Salmonella enterica*.

Download File: [Salmonella_enterica.txt](#)

Section 2

Code Challenge 5: Implement `MedianString()` Algorithm .

- **Input:** An integer k , followed by a space-separated collection of strings Dna .
 - **Output:** A k -mer *Pattern* that minimizes $d(Pattern, Dna)$ among all possible choices of k -mers. (If there are multiple such strings *Pattern*, then you may return any one.)
- # Create your median_string function, along with any subroutines you need
- ```
def median_string(dna: list[str], k: int) -> str:
 pass
```
- 

**Sample Input:**

```
3
AAATTGACGCAT GACGACCACGTT CGTCAGCGCCTG GCTGAGCACCGG
AGTACGGGACAG
```

---

**Sample Output:**

```
ACG
```

---

**Code Challenge 6:** Implement `GreedyMotifSearch()` with pseudocounts.

- **Input:** Integers  $k$  and  $t$ , followed by a space-separated collection of strings  $Dna$ .
  - **Output:** A collection of strings *BestMotifs* resulting from applying `GreedyMotifSearch( $Dna, k, t$ )` with pseudocounts. If at any step you find more than one *Profile*-most probable  $k$ -mer in a given string, use the one occurring first.
- # Create your greedy\_motif\_search\_pseudocounts function, along with any subroutines you need
- ```
def greedy_motif_search_pseudocounts(dna: list[str], k: int, t: int) -> list[str]:
    pass
```
-

Sample Input:

```
3 5
GGCGTTCAGGCA AAGAATCAGTCA CAAGGAGTTCGC CACGTCAATCAC
CAATAATATTTCG
```

Sample Output:

```
TTC ATC TTC ATC TTC
```

Section 3

Code Challenge 7: Solve the Overlap Graph Problem.

- **Input:** A collection *Patterns* of *k*-mers.
 - **Output:** The overlap graph *Overlap(Patterns)*, in the form of an adjacency list
Create your `overlap_graph` function, along with any subroutines you need
- ```
def overlap_graph(patterns: List[str]) -> Dict[str, List[str]]:
 pass
```
- 

**Sample Input:**

AAG AGA ATT CTA CTC GAT TAC TCT TCT TTC

---

**Sample Output:**

AAG: AGA  
AGA: GAT  
ATT: TTC  
CTA: TAC  
CTC: TCT  
GAT: ATT  
TCT: CTA CTC  
TTC: TCT

**Code Challenge 8:** Solve the de Bruijn Graph from *k*-mers Problem.

**DeBruijn Graph from *k*-mers Problem:** Construct the de Bruijn graph from a set of *k*-mers.

- **Input:** A collection of *k*-mers *Patterns*.
  - **Output:** The adjacency list of the de Bruijn graph *DeBruijn(Patterns)*.  
# Create your `de_bruijn_kmers` function, along with any subroutines you need
- ```
def de_bruijn_kmers(k_mers: List[str]) -> Dict[str, List[str]]:  
    pass
```
-

Sample Input:

GAGG CAGG GGGG GGGA CAGG AGGG GGAG

Sample Output:

AGG: GGG
CAG: AGG AGG
GAG: AGG
GGA: GAG
GGG: GGA GGG

Code Challenge 9: Solve the String Reconstruction from Read-Pairs Problem.

- **Input:** Integers k and d followed by a collection of paired k -mers *PairedReads*.
 - **Output:** A string *Text* with (k, d) -mer composition equal to *PairedReads*.
Create your StringReconstructionReadPairs function, along with any subroutines you need
- ```
def StringReconstructionReadPairs(PairedReads: List[Tuple[str, str]], k: int, d: int) -> str:
 pass
```
- 

**Sample Input:**

```
4 2
ACAC|CTCT ACAT|CTCA CACA|TCTC GACA|TCTC
```

---

**Sample Output:**

```
GACACATCTCTCA
```